

ICT703 Programming

Week 10 Review of Pandas

WARNING NOTICE

Commonwealth of Australia | Copyright Regulations 1969

WARNING

This material has been copied and communicated to you by or on behalf of the University of the Sunshine Coast pursuant to Part VA or Part VB of the Copyright Act 1968 (the Act).

The material in this communication may be subject to copyright under the Act.
Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.



Learning Objectives

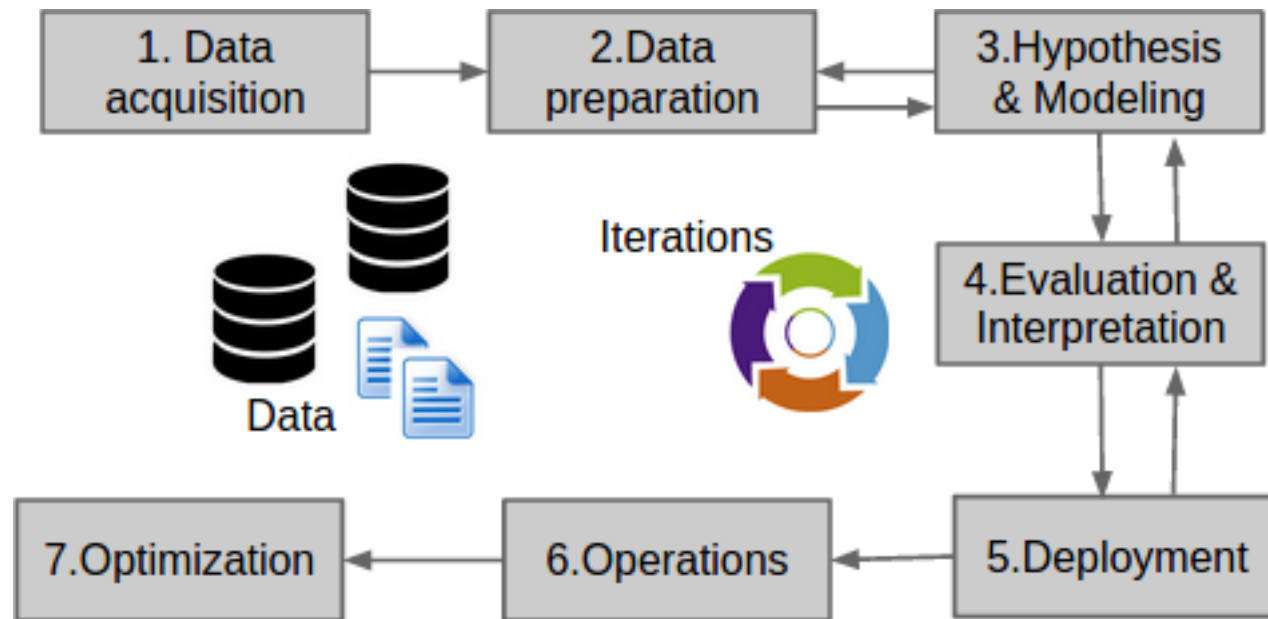
- See how to do deeper analysis on more complex data
- Learn how to *join* multiple files into one Pandas DataFrame
- Learn some helpful new DataFrame methods for large data:
 - `types`, `shape`, `columns`, `describe()`, `head()`, `tail()`, `pd.unique(...)`
- Be able to filter, sort, and group rows to create better graphs
- Understand referencing versus copying data?
- Be able to choose between different ways of handling missing data
- `iloc` versus `loc` versus `ix`?

We want to analyse Aust. Road Deaths

The kinds of 'road fatalities' questions we want to explore:

- Which roads are most dangerous (low-speed city roads, or highways)?
- What kinds of vehicles are most dangerous?
- How often are heavy vehicles (buses/trucks etc.) involved in crashes?
- Are there gender differences in crash rates or fatality rates?
- Does it matter whether someone is the driver or passenger?
- What age-group have the most crashes/fatalities?
- etc. etc.

The Data Science Life Cycle



<http://www.datasciencecentral.com/profiles/blogs/the-data-science-project-lifecycle>

Acquiring the Data

- Our usual Pandas setup:

```
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt
```

- Read the two CSV files using Pandas:

```
crashes = pd.read_csv("Fatal_Crashes_Oct_2016_II.csv")  
deaths = pd.read_csv("Fatalities_Oct_2016_II.csv")
```

Data Prep.1: Viewing our Data

`crashes.head()`

shows first 5 rows

	CrashID	State	Day	Month	Year	Hour	Minute	Crash_Type	Number_of_Fatalities	Bus_Involvement	Heavy_Rigid_Truck_Involvement
0	4201601010001	SA	1	1	2016	1	0	Single vehicle	1	No	No
1	2201601010001	VIC	1	1	2016	20	30	Single vehicle	1	No	No
2	1201601020001	NSW	2	1	2016	0	30	Single vehicle	1	No	No
3	5201601020001	WA	2	1	2016	17	20	Multiple vehicle	1	No	No
4	6201601020001	TAS	2	1	2016	19	58	Multiple vehicle	2	No	No

- Also:

`crashes.tail()`

shows last 5 rows

`crashes.dtypes`

shows the *type* of each column

Viewing a column

In: `crashes.State.describe()`

shows summary of values in this column

Out: count 42316
unique 8
top NSW
freq 13159
Name: State, dtype: object

In: `crashes.State.values`

Out: `array(['SA', 'VIC', 'NSW', ..., 'QLD', 'SA', 'VIC'], dtype=object)` # ALL the values! Duplicates!

In: `crashes.State.unique()`

Out: `array(['SA', 'VIC', 'NSW', 'WA', 'TAS', 'QLD', 'NT', 'ACT'], dtype=object)`
Good. No missing values. (No NaN values)

Viewing a column 2

Let's view a summary of the 'Crash_Type' column

In: `crashes.Crash_Type.value_counts()`

Out: Single vehicle 17818
Multiple vehicle 17125
Pedestrian 7373

- Only three kinds of crash type
- Weird that " Multiple Vehicle" starts with a space!
 - Seems to have happened during translation from Excel sheets to one CSV file.
 - We will *clean up* this later if it becomes a problem

Viewing a column 3

Let's view a summary of the 'Speed_Limit' column

In: `crashes.Speed_Limit.value_counts()`

Out:	100	14052
	60	12054
	80	4824
	110	4475
	50	2140
	70	2068
	90	900
		685
	900	519
	40	238
	75	233
	130	72
	20	20
	10	16
	30	12
	25	5
	888	1
	8	1
	45	1

Conclusions:

We need to do some data cleaning:

- remove the 3 singleton rows
- remove the 685 rows with missing speed limits
- investigate the 900kmh speed limit!
 - (After googling...)
 - NT had no speed limit on Stuart Highway until 2007! Then 130kmh

Viewing a column 4

In: `crashes[crashes.Speed_Limit == '900'].Year.value_counts().sort_index()`

Out:

1989	32
1990	31
1991	42
...	
2003	27
2004	19
2005	26
2006	14
2007	2

- So the speed limit did change in 2007!
- The '900' was not an error.
- It means 'unlimited speed zone'

Data Prep.2: Dealing with Missing Values

There are several strategies we can use for missing values

1. Leave the missing values there?
 - many Pandas methods will ignore NaN values
2. Replace the missing values by some estimated value:
 1. zero?
 2. a maximum value?
 3. the mean?
 - WARNING: this will change our results and graphs
 - We need to make sure such changes are sensible, and document them
3. Drop the rows that have missing values?
 - how many rows are we dropping?
 - How does this affect our results?

Replacing missing values

- The `fillna(...)` method gives lots of ways of replacing NaN values
 - Eg. `crashes.Speed_Limit = crashes.Speed_Limit.fillna(0)`
 - Or: `crashes.Speed_Limit.fillna(0, inplace=True)`
 - But this does not help with our Speed_Limit column. It has " ", not NaN.
 - Search for " " or use `isspace()`
- To replace a given value (" ") we select all those cells, then replace
 - `bad_speeds = crashes[crashes.Speed_Limit == " "].index`
 - `crashes.loc[bad_speeds, "Speed_Limit"] = 0`

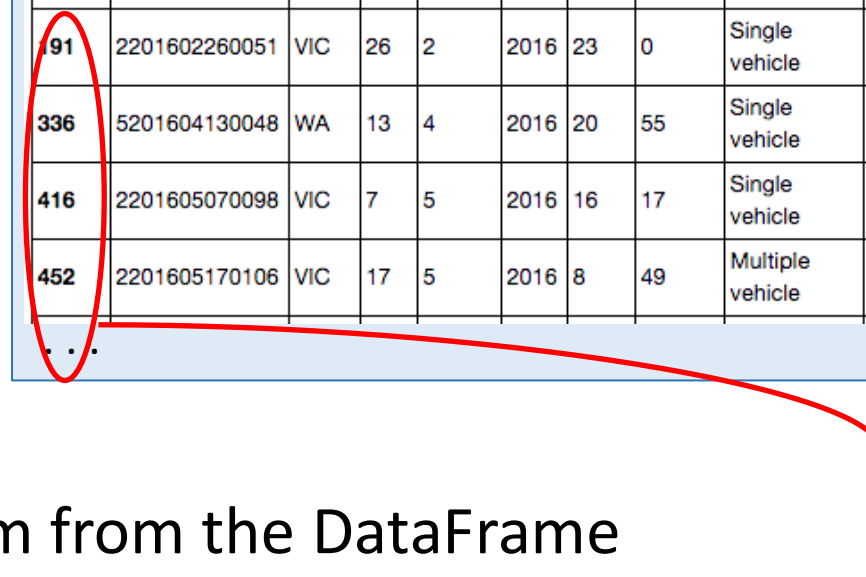
Dropping the missing values

1. find the bad rows

In: `missing = crashes[crashes.Speed_Limit == " "]`

Out:

	CrashID	State	Day	Month	Year	Hour	Minute	Crash_Type	Articulated_Truck_Involvement	Speed_Limit
191	2201602260051	VIC	26	2	2016	23	0	Single vehicle	No	
336	5201604130048	WA	13	4	2016	20	55	Single vehicle	No	
416	2201605070098	VIC	7	5	2016	16	17	Single vehicle	No	
452	2201605170106	VIC	17	5	2016	8	49	Multiple vehicle	No	
...										



2. drop them from the DataFrame

In: `crashes2 = crashes.drop(missing.index)`
`crashes2.shape`

Out: `(41631, 13)` # yes, 685 rows gone

Transforming data types

- We can change the type of a column using the **astype(...)** method
 - change Speed_Limit column from string to int (now all rows are numeric)

```
In:     crashes2.Speed_Limit = crashes2.Speed_Limit.astype(np.int)
```

- Now we can sort the speed limits properly

```
In:     crashes2.Speed_Limit.value_counts().sort_index()
```

```
Out:    10      16
      ...
      80    4824
      90     900
     100   14052
     110   4475
     130     72
     900    519
```

Data Prep.3: Merging Data

- The table has several same-name columns, but we merge on CrashID

```
data = crashes2.merge(deaths, on="CrashID",
                      suffixes=("", "_2"))
```

crashes3

	CrashID	State	Day	Month	Year	Hour	Minute	Crash_Type	Num
0	4201601010001	SA	1	1	2016	1	0	Single vehicle	1
1	2201601010001	VIC	1	1	2016	20	30	Single vehicle	1
2	1201601020001	NSW	2	1	2016	0	30	Single vehicle	1

deaths

	CrashID	State	Day	Month	Year	Hour	Minute	Crash_Type
0	4201601010001	SA	1	1	2016	1	0	Single
1	2201601010001	VIC	1	1	2016	20	30	Single
2	1201601020001	NSW	2	1	2016	0	30	Single
3	5201601020001	WA	2	1	2016	17	20	Multiple

data

	CrashID	State	Day	Month	Year	Hour	Minute	Crash_Type	Num	...	...	Heavy_Rigid_Truck_Involvement_2	Articulated_Truck_Involvement_2	Speed_Limit_2	Road_User	Gender	Age
0	4201601010001	SA	1	1	2016	1	0	Single vehicle	1	...	...	No	No	110	Driver	Male	40
1	2201601010001	VIC	1	1	2016	20	30	Single vehicle	1	...	...	No	No	80	Motor cycle rider	Male	25
2	1201601020001	NSW	2	1	2016	0	30	Single vehicle	1	...	...	No	No	100	Passenger	Male	18

Data Prep.4: Reshaping Data

- We can get rid of the duplicate columns

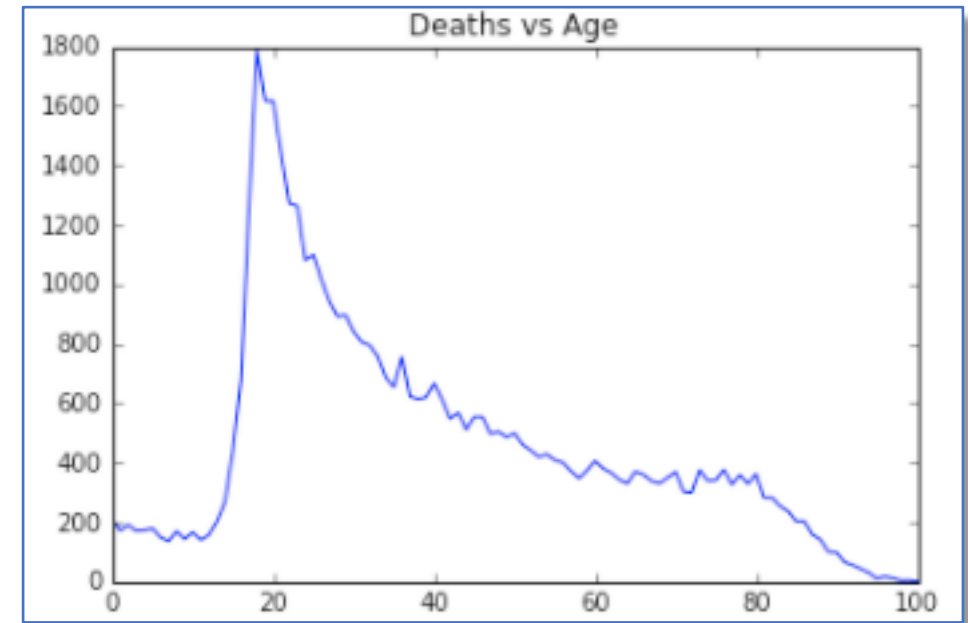
```
dup_cols = [col for col in data.columns.values if col.endswith("_2")]  
data2 = data.drop(dup_cols, axis=1)
```

- An ideal DataFrame shape follows the "Tidy Data" philosophy:
 1. Each column is a **variable** that we are observing
 2. Each row is one **observation** of those variables
 3. Each table (DataFrame) is a complete **observational unit**.(This is Codd's 3rd normal form, in database terms)
More details: <http://vita.had.co.nz/papers/tidy-data.pdf>
- To reshape a DataFrame: use melt(...) and pivot(...)
 - But in our example, 'data2' is already in Tidy Data form 😊

Hypotheses.1: Age matters?

- Let us investigate fatalities versus age
 - Each row in data2 is one fatality, and has an age column
`data2.Age.value_counts().sort_index().plot(kind="line", title="Deaths vs Age")`

- This raises more questions:
 - Is this *gender-dependent*?
 - Is it just the *driver* age that matter?
 - Maybe this is better in *recent years*?
- To answer these, we need *data grouping*

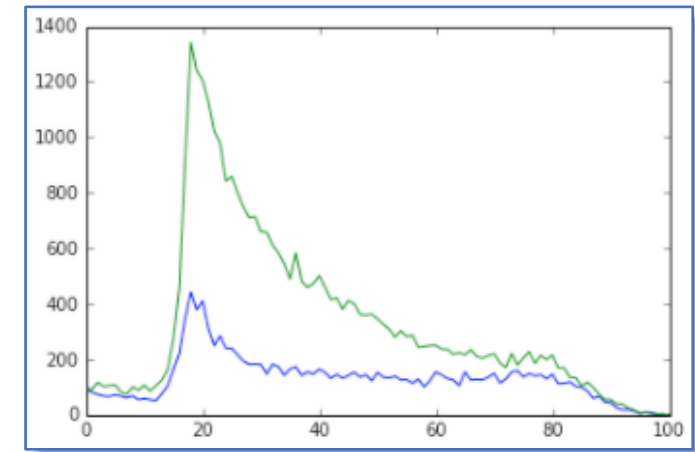
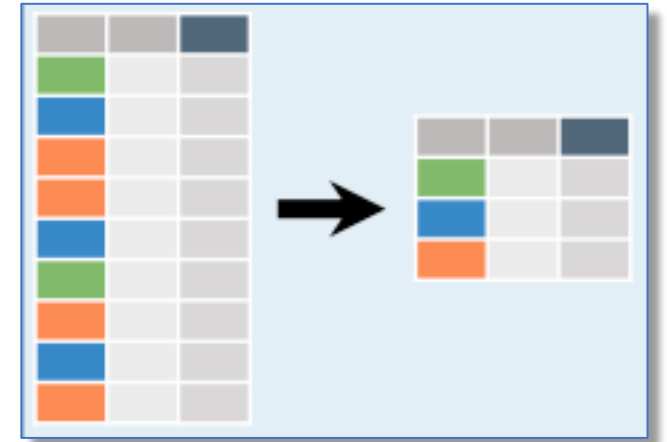


Data Grouping

```
sex_data = data2['sex']
# summary statistics for all numeric
columns by sex
sorted_data.describe()
# provide the mean for each
numeric column by sex
sorted_data.mean()
```

- **groupby(...)** supports many kinds of grouping
 - `groupby(column)` - by a single *column*
 - `groupby([col1,col2])` – by *col1*, then by *col2*
 - `groupby(function)` – by *function* applied to indexes
- `groupby` returns a `GroupBy` object
 - we can iterate over this object and get (key,table) pairs

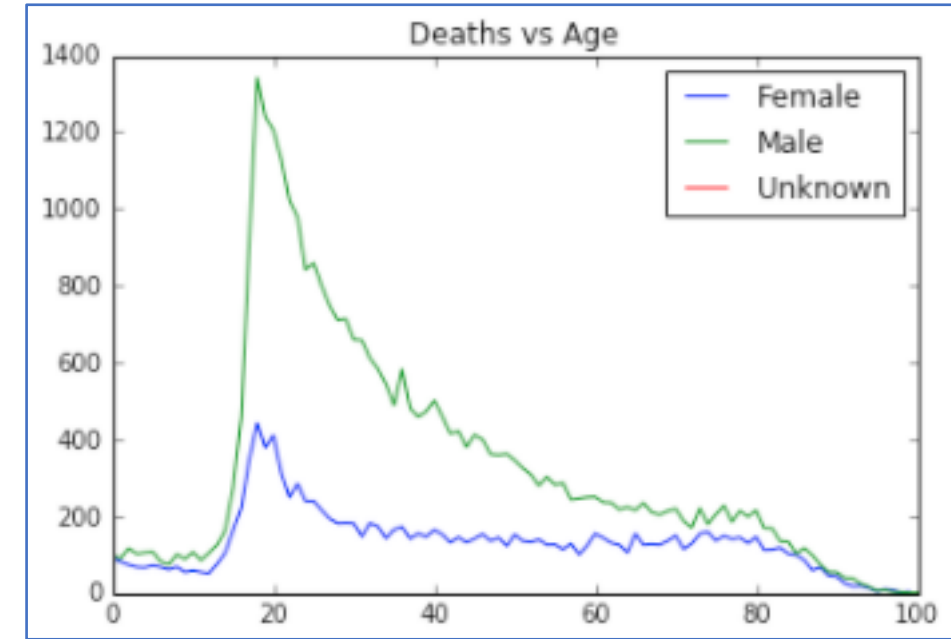
```
for gender,table in data2.groupby("Gender"):
    table.Age.value_counts().sort_index().plot.line()
```
 - Which line is Male? This needs a legend...



Data Grouping 2

- As we loop, we record the gender keys

```
genders = []  
for gender, table in data2.groupby("Gender"):  
    table.Age.value_counts().sort_index().plot.line()  
    genders.append(gender)  
plt.legend(genders)  
plt.title("Deaths vs Age")
```



Evaluation and Interpretation

- There are many more experiments we could do...
- Is it valid to make conclusions about death/risk *rates*?
 - No, not really: We need to know the background demographics & driving patterns
 - How many 20-year-olds drive (and how far), compared to 60-year-olds?
 - Do males just drive more than females?
 - Very difficult to get this background data
 - Google Maps **Traffic Layer** has some anonymous real-time road usage data
 - But we want long-term road usage statistics, broken down by demographics
 - [ABS Survey of Motor Vehicle Use](#) has vehicles/state and km/vehicle stats. This is a start...
- Data Interpretation is hard
 - you need some statistics background as well as data wrangling!

Summary (Pandas)

- Useful methods for viewing tables/columns

`head(), tail(), describe(), unique(), value_counts(), sum(), mean(), max(), std(), ...`

- Useful methods for replacing missing/bad data values

`fillna(...), dropna(), ispace()`

- DataFrame combining

`df1.merge(df2, on="col")`

- DataFrame grouping

`df1.groupby(by="col")`

- For more docs, see <http://pandas.pydata.org> & Pandas Cheat Sheet